

Introduction to Python and Pandas

IN5148: Statistics and Data Science with Applications in
Engineering

Alan R. Vazquez
Department of Industrial Engineering

Agenda

1. Introduction to Python
2. Reading data with Python
3. Data manipulation with pandas

Introduction to Python

Python

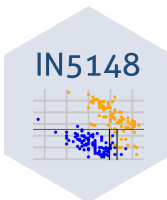
- A versatile programming language.
- It is free!
- It is widely used for data cleaning, data visualization, and data modelling.
- It can be extended with packages (libraries) developed by other users.



Google Colab

Google's free cloud collaboration platform for creating Python documents.

- Run Python and collaborate on Jupyter notebooks for free.
- Harness the power of GPUs for free to accelerate your data science projects.
- Easily save and upload your notebooks to Google Drive.



Let's try a command in Python

What do you think will happen if we run this command?

```
1 print("Hello world!")
```

Hello world!

Let's try another command

What do you think will happen if we run this command?

```
1 sum([1, 5, 10])
```

16

Use Python as a basic calculator

1 5 + 1

6

1 10 - 3

7

1 2 * 4

8

1 9 / 3

3.0

Comments

Sometimes we write things in the coding window that we want Python to ignore. These are called comments and start with `#`.

Python will ignore the comments and just execute the code.

```
1 # you can put whatever after #  
2 # for example... blah blah blah
```

Introduction to functions in Python

One of the best things about Python is that there are many built-in commands you can use. These are called *functions*.

Functions have two basic parts:

- The first part is the name of the function (for example, `sum`).
- The second part is the input to the function, which goes inside the parentheses (`sum([1, 5, 15])`).

Python is strict

Python, like all programming languages, is very strict. For example, if you write

```
1 sum([1, 100])
```

101

it will tell you the answer, 101.

But if you write

```
1 Sum([1, 100])
```

NameError

Traceback (most recent call last)

Cell In[11], line 1

----> 1 Sum([1, 100])

NameError: name 'Sum' is not defined

Save your work in Python objects

Virtually anything, including the results of any Python function, can be saved in an *object*.

This is accomplished by using an assignment operator, which can be an equals symbol (=).

You can make up any name you want for a Python object. However, there are two basic rules for this:

1. It **must** be different from a function name in Python.
2. It should be as specific as possible.

For example

```
1 # This code will assign the number 18
2 # to the object called my_favorite_number
3
4 my_favorite_number = 18
```

After running this code, nothing happens. But if we run the object on its own, we can see what's inside it.

```
1 my_favorite_number
```

18

You can also use `print(my_favorite_number)`.

Lists

So far we have used Python objects to store a single number. But in statistics we are dealing with variation, which by definition needs more than one number.

A Python object can also store a complete set of numbers, called a *list*.

You can think of a list as a vector of numbers (or values).

The `[]` command can be used to combine several individual values into a list.

For example

This code creates two vectors

```
1 my_list = [1, 2, 3, 4, 5]
2 my_list_2 = [10, 10, 10, 10, 10]
```

Let's see its content

```
1 my_list
```

```
[1, 2, 3, 4, 5]
```

```
1 my_list_2
```

```
[10, 10, 10, 10, 10]
```

Operations

We can do simple operations with vectors. For example, we can sum all the elements of a list.

```
1 my_list = [1, 2, 3, 4, 5]
2 sum(my_list)
```

15

Indexing

We can index a position in the vector using square brackets with a number like this: `[1]`.

So, if we wanted to print the contents of the first position in `my_list`, we could write

```
1 my_list[1]
```

2

An feature of Python is that the first element of a list or vector is indexed using the number 0.

```
1 my_list[0]
```

1

A little more about objects in Python

You can think of Python objects as containers that hold values.

A Python object can hold a single value, or it can hold a group of values (as in a vector).

So far, we've only put numbers into Python objects.

Python objects can actually contain three types of values: numbers, characters, and booleans.

Character values

Characters are made up of text, such as words or sentences. An example of a list with characters as elements is:

```
1 many_greetings = ["hi", "hello", "hola", "bonjour", "ni hao", "merhaba"]  
2 many_greetings
```

```
['hi', 'hello', 'hola', 'bonjour', 'ni hao', 'merhaba']
```

It is important to know that numbers can also be treated as characters, depending on the context.

For example, when 20 is enclosed in quotes ("20") it will be treated as a character value, even though it encloses a number in quotes.

Boolean values

Boolean values are `True` or `False`.

We may have a question like:

- Is the first element of the vector `many_greetings` "hola"?

We can ask Python to find out and return the answer `True` or `False`.

```
1 many_greetings[1] == "hola"
```

`False`

Logical operators

Most of the questions we ask Python to answer with `True` or `False` involve comparison operators like `>`, `<`, `>=`, `<=`, and `==`.

The double `==` sign checks whether two values are equal.

There is even a comparison operator to check whether values are *not* equal: `!=`.

For example, `5 != 3` is a `True` statement.

Common logical operators

- $>$ (*larger than*)
- $\geq$ (*larger than or equal to*)
- $<$ (*smaller than*)
- $\leq$ (*smaller than or equal to*)
- $==$ (*equal to*)
- $!=$ (*not equal to*)

Question

Read this code and predict its response. Then, run the code in Google Colab and validate if you were correct.

```
1 A = 1
2 B = 5
3 compare = A > B
4 compare
```

Programming culture: Trial and error

The best way to learn programming is to try things out and see what happens. Write some code, run it, and think about why it didn't work.

There are many ways to make small mistakes in programming (for example, typing a capital letter when a lowercase letter is needed).

We often have to find these mistakes through trial and error.

Python libraries

Libraries are the fundamental units of reproducible Python code. They include reusable Python functions, documentation describing how to use them, and sample data.

In this course, we will be working mostly with the following libraries:

- `pandas` for data manipulation
- `matplotlib` and `seaborn` for data visualization
- `statsmodels` and `scikit-learn` for data modelling

Reading data with Python

Data organization

In data science, we organize data into rows and columns.

	Condition	Age	Wt	Wt2
1	Uninformed	35	136	135.8
2	Uninformed	45	162	161.8
3	Informed	52	117	116.8
4	Informed	29	184	182.8
5	Uninformed	38	134	136.6
6	Informed	39	189	183.2

The rows are the sampled cases. In this example, the rows are housekeepers from different hotels. There are six rows, so there are six housekeepers in this data set.

Depending on the study, the rows could be people, states, couples, mice—any case you're taking a sample from to study.

The columns represent variables or attributes of each case that were measured.

	Condition	Age	Wt	Wt2
1	Uninformed	35	136	135.8
2	Uninformed	45	162	161.8
3	Informed	52	117	116.8
4	Informed	29	184	182.8
5	Uninformed	38	134	136.6
6	Informed	39	189	183.2

In this study, housekeepers were either informed or not that their daily work of cleaning hotel rooms was equivalent to getting adequate exercise for good health.

So one of the variables, **Condition**, indicates whether they were informed of this fact or not.

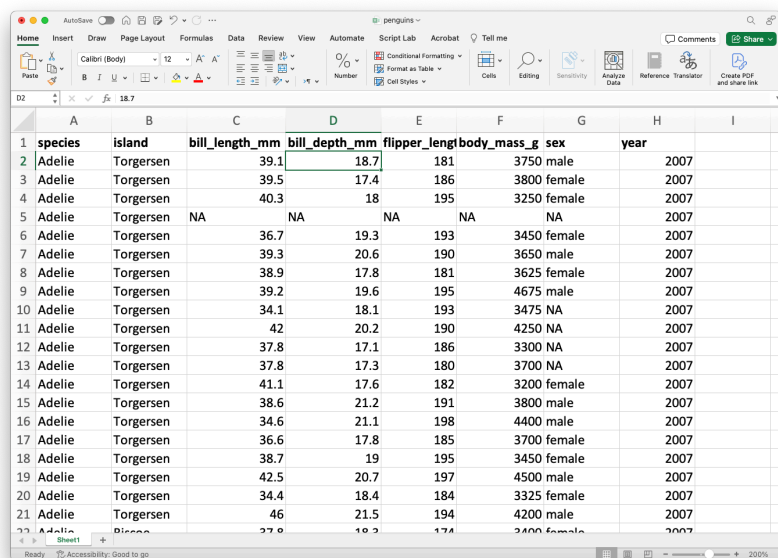
Other variables include the age of the housekeeper (**Age**), her weight before starting the study (**Wt**), and her weight at the end of the study (**Wt2**), measured four weeks later.

Therefore, the values in each row represent the values of that particular case in each of the variables measured.

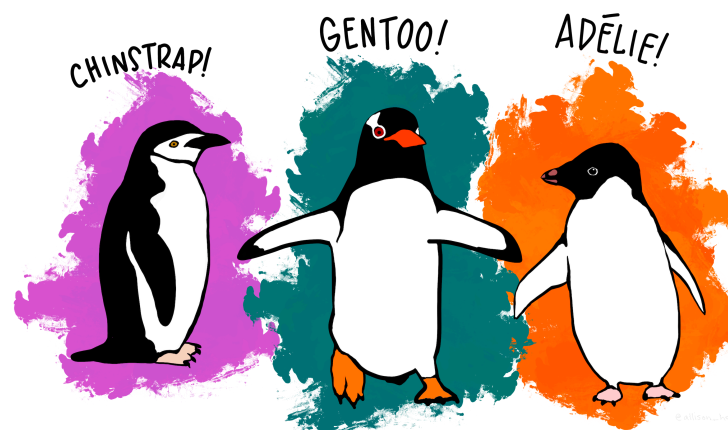
	Condition	Age	Wt	Wt2
1	Uninformed	35	136	135.8
2	Uninformed	45	162	161.8
3	Informed	52	117	116.8
4	Informed	29	184	182.8
5	Uninformed	38	134	136.6
6	Informed	39	189	183.2

Loading data in Python

In this course, we will assume that data is stored in an Excel file with the above organization. As an example, let's use the file `penguins.xlsx`.

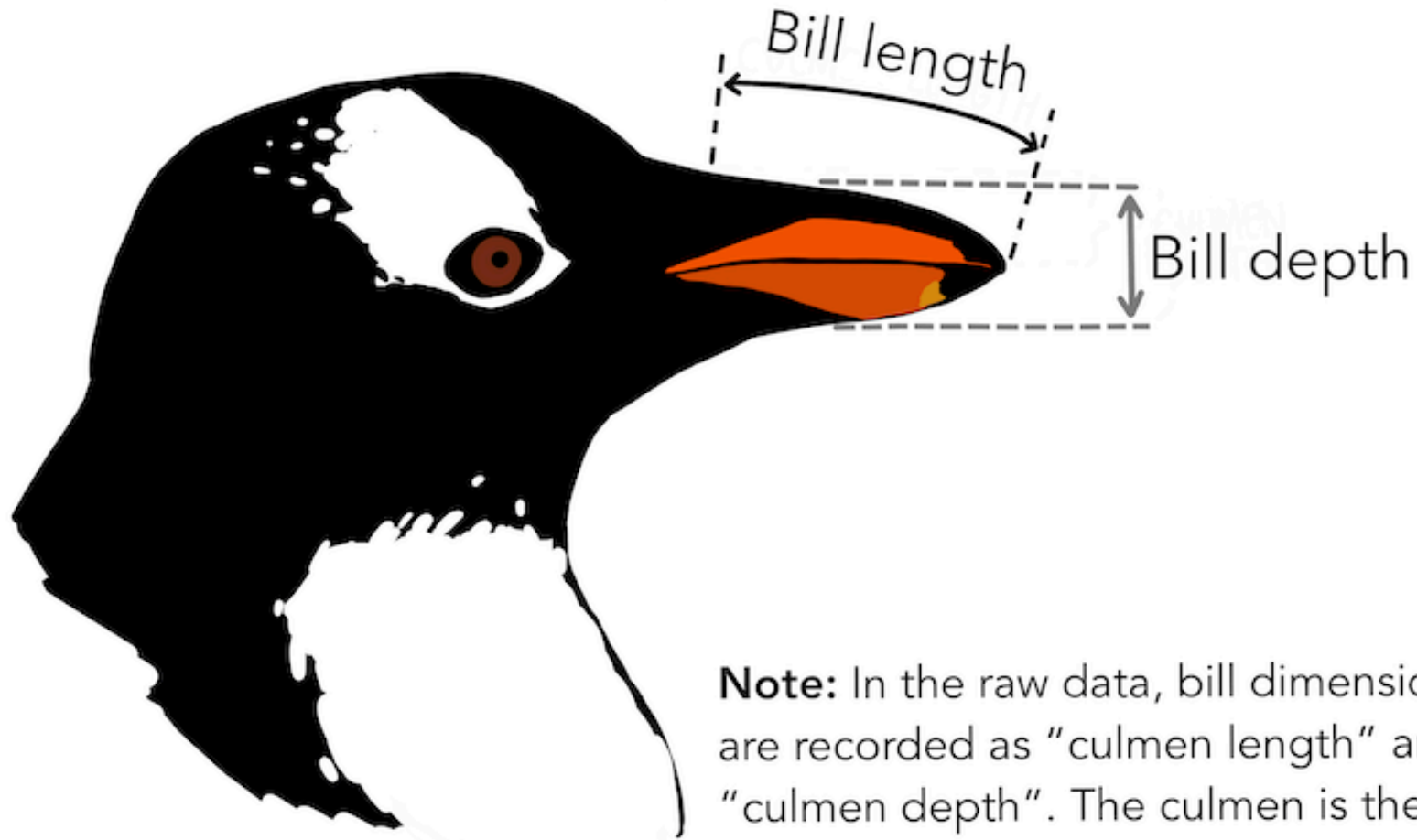


	A	B	C	D	E	F	G	H	I
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year	
2	Adelie	Torgersen	39.1	18.7	181	3750	male	2007	
3	Adelie	Torgersen	39.5	17.4	186	3800	female	2007	
4	Adelie	Torgersen	40.3	18	195	3250	female	2007	
5	Adelie	Torgersen	NA	NA	NA	NA	NA	2007	
6	Adelie	Torgersen	36.7	19.3	193	3450	female	2007	
7	Adelie	Torgersen	39.3	20.6	190	3650	male	2007	
8	Adelie	Torgersen	38.9	17.8	181	3625	female	2007	
9	Adelie	Torgersen	39.2	19.6	195	4675	male	2007	
10	Adelie	Torgersen	34.1	18.1	193	3475	NA	2007	
11	Adelie	Torgersen	42	20.2	190	4250	NA	2007	
12	Adelie	Torgersen	37.8	17.1	186	3300	NA	2007	
13	Adelie	Torgersen	37.8	17.3	180	3700	NA	2007	
14	Adelie	Torgersen	41.1	17.6	182	3200	female	2007	
15	Adelie	Torgersen	38.6	21.2	191	3800	male	2007	
16	Adelie	Torgersen	34.6	21.1	198	4400	male	2007	
17	Adelie	Torgersen	36.6	17.8	185	3700	female	2007	
18	Adelie	Torgersen	38.7	19	195	3450	female	2007	
19	Adelie	Torgersen	42.5	20.7	197	4500	male	2007	
20	Adelie	Torgersen	34.4	18.4	184	3325	female	2007	
21	Adelie	Torgersen	46	21.5	194	4200	male	2007	



The file must be previously uploaded to Google Colab.

The dataset `penguins.xlsx` contains data from penguins living in three islands.



Note: In the raw data, bill dimensions are recorded as "culmen length" and "culmen depth". The culmen is the dorsal ridge atop the bill.

pandas library



- **pandas** is an open-source Python library for data manipulation and analysis.
- It is built on top of **numpy** for high-performance data operations..
- It allows the user to import, clean, transform, and analyze data efficiently
- <https://pandas.pydata.org/>

Importing pandas

Fortunately, the **pandas** library is already pre-installed in Google Colab.

However, we need to inform Google Colab that we want to use **pandas** and its functions using the following command:

```
1 import pandas as pd
```

The command **as pd** allows us to have a short name for **pandas**. To use a function of **pandas**, we use the command **pd.function()**.

Loading data using pandas

The following code shows how to read the data in the file “penguins.xlsx” into Python.

```
1 # Load the Excel file into a pandas DataFrame.  
2 penguins_data = pd.read_excel("penguins.xlsx")
```

The function head()

The function `head()` allows you to print the first rows of a pandas data frame.

```
1 # Print the first 4 rows of the dataset.  
2 penguins_data.head(4)
```

	species	island	bill_length_mm	bill_depth_mm
0	Adelie	Torgersen	39.1	18.7
1	Adelie	Torgersen	39.5	17.4
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN

Indexing variables a dataset

We can select a specific variables of a data frame using the syntax below.

```
1 penguins_data['bill_length_mm']
```

```
0      39.1
1      39.5
2      40.3
3       NaN
4      36.7
```

```
...
```

```
339     55.8
340     43.5
341     49.6
342     50.8
343     50.2
```

```
Name: bill_length_mm, Length: 344, dtype: float64
```

Here, we selected the variable `bill_length_mm` in the `penguins_data` dataset.

To index multiple variables of a data frame, we put the names of the variables in a list object. For example, we select `bill_length_mm`, `species`, and `island` as follows:

```
1 sub_penguins_data = penguins_data[ ['bill_length_mm', 'species', 'island']  
2 sub_penguins_data.head()
```

	<code>bill_length_mm</code>	<code>species</code>	<code>island</code>
0	39.1	Adelie	Torgersen
1	39.5	Adelie	Torgersen
2	40.3	Adelie	Torgersen
3	NaN	Adelie	Torgersen
4	36.7	Adelie	Torgersen

Indexing rows

To index rows in a dataset, we use the argument `loc` from **pandas**. For example, we select the rows 3 to 6 of the `penguins_dataset` dataset:

```
1 rows_penguins_data = penguins_data.loc[2:5]
2 rows_penguins_data
```

	species	island	bill_length_mm	bill_depth_mm
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN
4	Adelie	Torgersen	36.7	19.3
5	Adelie	Torgersen	39.3	20.6

```
1 rows_penguins_data
```

	species	island	bill_length_mm	bill_depth_mm
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN
4	Adelie	Torgersen	36.7	19.3
5	Adelie	Torgersen	39.3	20.6

Note that the index 2 and 5 refer to observations 3 and 7, respectively, in the dataset. This is because the first index in Python is 0.

Indexing rows and columns

Using `loc`, we can also retrieve a subset from the dataset by selecting specific columns and rows.

```
1 sub_rows_pdata = penguins_data.loc[2:5, ['bill_length_mm', 'species', 'island']]
2 sub_rows_pdata
```

	bill_length_mm	species	island
2	40.3	Adelie	Torgersen
3	NaN	Adelie	Torgersen
4	36.7	Adelie	Torgersen
5	39.3	Adelie	Torgersen

Data manipulation with pandas

Chaining operations with pandas

One of the most important techniques in pandas is **chaining**, which allows for cleaner and more readable data manipulation.

The general structure of chaining looks like this:

Key pandas methods

pandas provides methods or functions to solve common data manipulation tasks:

- `.filter()` selects specific columns or rows.
- `.query()` filters observations based on conditions.
- `.assign()` adds new variables that are functions of existing variables.
- `.sort_values()` changes the order of rows.
- `.agg()` reduces multiple values to a single numerical summary.

A QUICK INTRODUCTION TO DATA MANIPULATION WITH PANDAS

```
(dataframe  
    .filter(...)  
    .query(...)  
    .assign(...)  
    .sort_values(...)  
    .agg(...)  
)
```

To practice, we will use the dataset [penguins_data](#).

Example

Let's load the dataset and the **pandas** library.

```
1 import pandas as pd
2
3 # Load the Excel file into a pandas DataFrame.
4 penguins_data = pd.read_excel("penguins.xlsx")
5
6 # Preview the dataset.
7 penguins_data.head(4)
```

	species	island	bill_length_mm	bill_depth_mm
0	Adelie	Torgersen	39.1	18.7
1	Adelie	Torgersen	39.5	17.4
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN

Selecting columns with `.filter()`

Select the columns `species`, `body_mass_g` and `sex`.

```
1 (penguins_data  
2   .filter(["species", "body_mass_g", "sex"], axis = 1)  
3 ).head()
```

	species	body_mass_g	sex
0	Adelie	3750.0	male
1	Adelie	3800.0	female
2	Adelie	3250.0	female
3	Adelie	NaN	NaN
4	Adelie	3450.0	female

The `axis` argument tells `.filter()` whether to select rows (`0`) or columns (`1`) from the dataframe.

```
1 (penguins_data
2   .filter(["species", "body_mass_g", "sex"], axis = 1)
3 ).head()
```

The `.head()` command allows us to print the first six rows of the newly produced dataframe. We **must** remove it to have the entire new dataframe.

We can also use `.filter()` to select rows too. To this end, we set `axis = 1`. We can select specific rows, such as 0 and 10.

```
1 (penguins_data
2   .filter([0, 10], axis = 0)
3 )
```

	species	island	bill_length_mm	bill_depth_mm
0	Adelie	Torgersen	39.1	18.7
10	Adelie	Torgersen	37.8	17.1

Or, we can select a set of rows using the function `range()`. For example, let's select the first 5 rows.

```
1 (penguins_data
2   .filter(range(5), axis = 0)
3 )
```

	species	island	bill_length_mm	bill_depth_mm
0	Adelie	Torgersen	39.1	18.7
1	Adelie	Torgersen	39.5	17.4
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN
4	Adelie	Torgersen	36.7	19.3

Filtering rows with `.query()`

An alternative way of selecting rows is `.query()`. Compared to `.filter()`, `.query()` allows us to filter the data using statements or *queries* involving the variables.

For example, let's filter the data for the species "Gentoo."

```
1 (penguins_data
2   .query("species == 'Gentoo'")
3   )
```

```
1 (penguins_data  
2   .query("species == 'Gentoo'")  
3 ).head()
```

	species	island	bill_length_mm	bill_depth_mm
152	Gentoo	Biscoe	46.1	13.2
153	Gentoo	Biscoe	50.0	16.3
154	Gentoo	Biscoe	48.7	14.1
155	Gentoo	Biscoe	50.0	15.2
156	Gentoo	Biscoe	47.6	14.5

We can also filter the data to get penguins with a body mass greater than 5000g.

```
1 (penguins_data
2   .query("body_mass_g > 5000")
3   ).head()
```

	species	island	bill_length_mm	bill_depth_mm
153	Gentoo	Biscoe	50.0	16.3
155	Gentoo	Biscoe	50.0	15.2
156	Gentoo	Biscoe	47.6	14.5
159	Gentoo	Biscoe	46.7	15.3
161	Gentoo	Biscoe	46.8	15.4

We can even **combine** `.filter()` and `.query()`. For example, let's select the columns `species`, `body_mass_g` and `sex`, then filter the data for the “Gentoo” species.

```
1 (penguins_data
2   .filter(["species", "body_mass_g", "sex"], axis = 1)
3   .query("species == 'Gentoo'")
4   ).head(4)
```

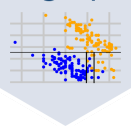
	species	body_mass_g	sex
152	Gentoo	4500.0	female
153	Gentoo	5700.0	male
154	Gentoo	4450.0	female
155	Gentoo	5700.0	male

Create new columns with `.assign()`

With `.assign()`, we can create new columns (variables) that are functions of existing ones. This function uses a special Python keyword called `lambda`. Technically, this keyword defines an *anonymous* function.

For example, we create a new variable `LDRatio` equaling the ratio of `bill_length_mm` and `bill_depth_mm`.

```
1 (penguins_data
2   .assign(LDRatio = lambda df: df["bill_length_mm"] / df["bill_depth_mm"])
3 )
```



In this code, the `df` after `lambda` indicates that the dataframe (`penguins_data`) will be referred to as `df` inside the function. The colon `:` sets the start of the function.

```
1 (penguins_data
2   .assign(LDRatio = lambda df: df["bill_length_mm"] / df["bill_depth_mm"])
3 )
```

The code appends the new variable to the end of the resulting dataframe.

We can see the new variable using `.filter()`.

```
1 (penguins_data
2   .assign(LDRatio = lambda df: df["bill_length_mm"] / df["bill_depth_mm"])
3   .filter(["bill_length_mm", "bill_depth_mm", "LDRatio"], axis = 1)
4   ).head()
```

	bill_length_mm	bill_depth_mm	LDRatio
0	39.1	18.7	2.090909
1	39.5	17.4	2.270115
2	40.3	18.0	2.238889
3	NaN	NaN	NaN
4	36.7	19.3	1.901554

Sorting with `.sort_values()`

We can sort the data based on a column like `bill_length_mm`.

```
1 (penguins_data
2   .sort_values("bill_length_mm")
3 ).head(4)
```

	species	island	bill_length_mm	bill_depth_mr
142	Adelie	Dream	32.1	15.5
98	Adelie	Dream	33.1	16.1
70	Adelie	Torgersen	33.5	19.0
92	Adelie	Dream	34.0	17.1

To sort in descending order, use `ascending=False` inside `sort_values()`.

```
1 (penguins_data
2   .sort_values("bill_length_mm", ascending=False)
3   ).head()
```

	species	island	bill_length_mm	bill_depth_mm
185	Gentoo	Biscoe	59.6	17.0
293	Chinstrap	Dream	58.0	17.8
253	Gentoo	Biscoe	55.9	17.0
339	Chinstrap	Dream	55.8	19.8
267	Gentoo	Biscoe	55.1	16.0

Summarizing with `.agg()`

We can calculate summary statistics of the columns `bill_length_mm`, `bill_depth_mm`, and `body_mass_g`.

```
1 (penguins_data
2   .filter(["bill_length_mm", "bill_depth_mm", "body_mass_g"], axis = 1)
3   .agg(["mean"])
4 )
```

	bill_length_mm	bill_depth_mm	body_mass_g
mean	43.92193	17.15117	4201.754386

By default, `agg()` ignores missing values.

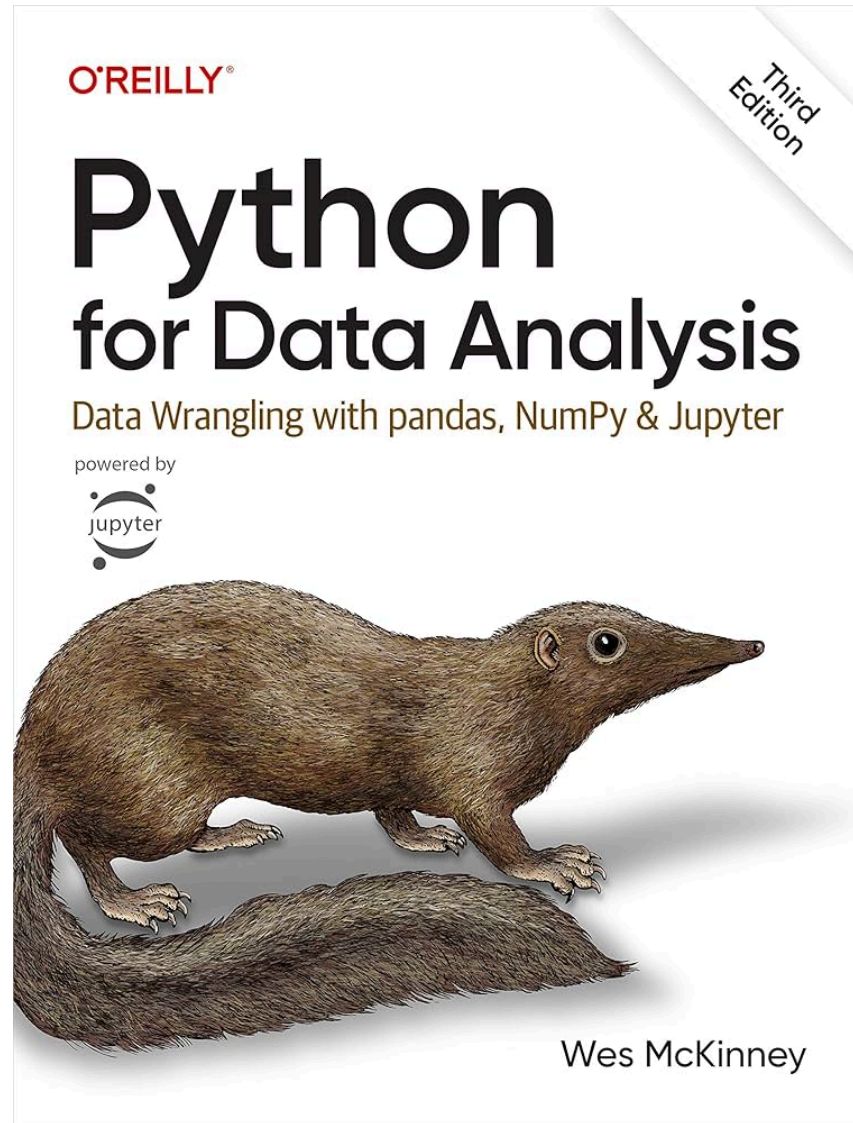
Saving results in new objects

After performing operations on our data, we can save the modified dataset as a new object.

```
1 mean_penguins_data = (penguins_data
2   .filter(["bill_length_mm", "bill_depth_mm", "body_mass_g"], axis = 1)
3   .agg(["mean"]))
4 )
5
6 mean_penguins_data
```

	bill_length_mm	bill_depth_mm	body_mass_g
mean	43.92193	17.15117	4201.754386

More on pandas



<https://wesmckinney.com/book/>

Return to main page